

Checkpoint 01

Segurança open source no pipeline DevSecOps

SAST · SCA · IaC Security · DAST

Anderson Huang RM 565920

Bruno Henrique RM 566277

Guyherme Miguel RM 562374

Luiz Brito RM 562192

Cloud Security: Automation e DevSecOps · FIAP · 2026



Introdução

Agenda

Abertura		3 min
As quatro ferramentas		10 min
Laboratório ao vivo		12 min
Comparativo e conclusão		3 min
Perguntas		2 min

As quatro frentes

	SAST	SCA	IaC Security	DAST
O que analisa	Código-fonte	Dependências	Arquivos de infraestrutura	Aplicação em execução
Estágio do pipeline	Code	Build	Code e Deploy	Test
Exige a aplicação em execução	Não	Não	Não	Sim

Posicionamento no pipeline

	Code	Build	Test	Deploy
SAST				
SCA				
IaC Security				
DAST				

SAST

Semgrep: o que é



Análise estática por correspondência de padrões sobre a árvore sintática do código.

Licença

LGPL-2.1

Escrito em

OCaml

Semgrep: como funciona

A regra se parece com o código que procura.

Regra · xss.yaml

```
rules:  
  - id: express-xss  
    pattern: res.send($A + req.query.$X)  
    languages: [javascript]  
    severity: ERROR
```

Código · app.js

```
7 app.get("/hello", (req, res) => {  
8   // VULNERAVEL  
9   res.send("Hello, " + req.query.name);
```

SAST

Semgrep: resultado

84 s

tempo de execução

223

regras avaliadas

2

achados

Encontrou XSS na linha 9 sem subir a aplicação.

SCA

Dependency-Check: o que é



Compara as dependências declaradas com a base pública de vulnerabilidades da NVD.

Licença

Apache-2.0

Escrito em

Java

Projeto desde

2012

Dependency-Check: como funciona

A base precisa ser baixada antes da primeira análise.



SCA

Dependency-Check: limitação

398.697

registros na base da NVD

A base da NVD exige chave de API e download demorado, o que torna a ferramenta dependente de rede e cache.

Checkov: o que é

checkov

Avalia políticas sobre um grafo de recursos de infraestrutura.

Licença

Apache-2.0

Escrito em

Python

Checkov: como funciona

Terraform · iac/main.tf

```
security_rule {  
  name           = "allow-ssh"  
  direction      = "Inbound"  
  access         = "Allow"  
  destination_port_range = "22"  
  source_address_prefix = "*"   
}
```

Veredito

CKV_AZURE_10

Reprovado

Acesso SSH liberado a partir da internet

Checkov: a limitação que encontramos

```
source_address_prefix = "*"

```

Reprovado

```
source_address_prefix = "0.0.0.0/0"

```

Aprovado

Mesma exposição, vereditos opostos. É um falso negativo.

DAST

ZAP: o que é



Scanner black-box que ataca a aplicação em execução.

Licença

Apache-2.0

Projeto desde

2010

DAST

ZAP: como funciona

1

Spider

descobre as URLs

2

Scan passivo

observa as respostas

3

Scan ativo

injeta payloads

Apenas o scan ativo encontra XSS.

DAST

ZAP: o gate por severidade

Script empacotado

3 alertas de risco alto

2

código de saída

Plano de automação

errorLevel: High

1

código de saída

Só a segunda forma quebra o build por criticidade.

Laboratório: como funciona

```
git clone https://github.com/org-cloud-security/cp01-devsecops.fiap.git
```

Fase 1

Checkov reprova

Fase 2

Correção do
Terraform

Fase 3

ZAP alerta risco alto

Fase 4

Correção da
aplicação

Fase 5

Encerrar o ambiente

A turma executa junto. Pré-requisitos: Docker, Docker Compose e Git.

Fase 1 e 2, Checkov

Fase 1 · Reprovado

`source_address_prefix = "*"`

1

código de saída

Fase 2 · Aprovado

`source_address_prefix = "10.0.0.0/24"`

0

código de saída

Uma linha alterada em `iac/main.tf`. Depois da correção: 4 checks aprovados, 0 reprovados.

Fase 3 e 4, ZAP

Fase 3 · Alerta de risco alto

```
res.send("Hello, " + req.query.name)
```

1

código de saída

Fase 4 · Plano bem-sucedido

```
res.type("text").send("Hello, " + req.query.name)
```

0

código de saída

Uma linha alterada em app/index.js: a resposta passa a ser texto puro.

Perguntas de verificação

1

Qual o ID do check que o Checkov reprovou, e quantos checks passaram depois da correção?

2

Quantos alertas de risco High o ZAP levantou antes da correção, e qual o CWE do XSS refletido?

Conclusão

Quadro comparativo

Ferramenta	Categoria	Licença	App no ar	Severidade	SARIF
 Semgrep	SAST	LGPL-2.1	Não	Sim	Sim
 OWASP	SCA	Apache-2.0	Não	Sim	Sim
 checkov	IaC	Apache-2.0	Não	Não	Sim
 ZAP	DAST	Apache-2.0	Sim	Sim	Sim

Conclusão

A convergência

Semgrep

lendo o código parado

ZAP

atacando a aplicação no ar

CWE-79

Cross-site scripting

O mesmo achado, por caminhos independentes.

Conclusão

Limitações que encontramos

Checkov

Não classifica severidade na versão gratuita.

ZAP

O código de saída dos scripts ignora severidade.

Checkov

A política CKV_AZURE_10 não reconhece notação CIDR.

ZAP

Alertas passivos variam entre execuções idênticas.

Conclusão

Recomendação de toolchain

Code	Build	Test	Deploy
			
Semgrep	Dependency-Check	ZAP, plano de automação	Checkov
Gate	Gate	Gate	Informativo

O gate por criticidade precisa ficar em ferramenta que atribui severidade real.

Encerramento

Perguntas

Repositório do laboratório

github.com/org-cloud-security/cp01-devsecops.fiap

Anderson Huang

Bruno Henrique

Guyllherme Miguel

Luiz Brito